AI ASSISTED CODING

ASSIGNMENT-12.4

HTNO: 2303A510D7

NAME: V.Sathwik reddy

BATCH: 14

Lab 12 – Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task 1: Bubble Sort for Ranking Exam Scores

Scenario

You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment.

Task Description

• Implement Bubble Sort in Python to sort a list of student scores.

• Use an AI tool to:

o Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes

o Identify early-termination conditions when the list becomes sorted

o Provide a brief time complexity analysis

Expected Outcome

• A Bubble Sort implementation with:

o AI-generated comments explaining the logic

o Clear explanation of best, average, and worst-case complexity

o Sample input/output showing sorted scores

PROMPT:

#Implement Bubble Sort in Python to sort a list of student scores.

#Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes

#Identify early-termination conditions when the list becomes sorted

#Provide a brief time complexity analysis and Show sample input and output in user input.

CODE:

lab(12.4) > t1.py > ...

```python
#Implement Bubble Sort in Python to sort a list of student scores.
#Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes
#Identify early-termination conditions when the list becomes sorted
#Provide a brief time complexity analysis and Show sample input and output in user input.
def bubble_sort(scores):
    """
    Sort a list of student scores using the Bubble Sort algorithm.

    Parameters:
    scores: A list of student scores to be sorted.

    Returns:
    A sorted list of student scores.
    """
    n = len(scores)
    # Traverse through all elements in the list
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements
            if scores[j] > scores[j+1]:
                # Swap if the element found is greater than the next element
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True  # Set the flag to True if a swap occurred
        # If no swapping occurred, the list is already sorted
        if not swapped:
            break  # Exit early if the list is already sorted
    return scores
# Example usage in user input
if __name__ == "__main__":
    # Get user input for student scores
    input_scores = input("Enter student scores separated by commas: ")
    # Convert the input string into a list of integers
    scores_list = [int(score.strip()) for score in input_scores.split(',')]
    # Sort the scores using bubble sort
```

OUTPUT:

```python
    sorted_scores = bubble_sort(scores_list)
    # Print the sorted list of scores
    print("Sorted student scores:", sorted_scores)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Python313/python.exe "c:
12.4)/t1.py"
Enter student scores separated by commas: 99,95,40,60,55,35
Sorted student scores: [35, 40, 55, 60, 95, 99]
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>
```

JUSTIFICATION:

Bubble Sort was implemented with early termination optimization.
The algorithm correctly sorted student scores after each internal assessment.
AI assistance helped in adding meaningful comments and improving performance by identifying the early-termination condition.
Although Bubble Sort has $O(n^2)$ complexity, it is suitable for small datasets.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario

You are maintaining an attendance system where student roll numbers

are already almost sorted, with only a few late updates.

Task Description

• Start with a Bubble Sort implementation.

• Ask AI to:

o Review the problem and suggest a more suitable sorting

algorithm

o Generate an Insertion Sort implementation

o Explain why Insertion Sort performs better on nearly

sorted data

• Compare execution behavior on nearly sorted input

Expected Outcome

• Two sorting implementations:

o Bubble Sort

o Insertion Sort

• AI-assisted explanation highlighting efficiency differences for

partially sorted datasets

PROMPT1:

#Given that attendance roll numbers are nearly sorted, start with a Bubble Sort implementation.

CODE:

```python
#Given that attendance roll numbers are nearly sorted, start with a Bubble Sort implementation.
def bubble_sort_attendance(roll_numbers):
    """
    Sort a list of attendance roll numbers using the Bubble Sort algorithm.

    Parameters:
    roll_numbers: A list of attendance roll numbers to be sorted.

    Returns:
    A sorted list of attendance roll numbers.
    """
    n = len(roll_numbers)
    # Traverse through all elements in the list
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements
            if roll_numbers[j] > roll_numbers[j+1]:
                # Swap if the element found is greater than the next element
                roll_numbers[j], roll_numbers[j+1] = roll_numbers[j+1], roll_numbers[j]
                swapped = True  # Set the flag to True if a swap occurred
        # If no swapping occurred, the list is already sorted
        if not swapped:
            break  # Exit early if the list is already sorted
    return roll_numbers
# Example usage in user input
if __name__ == "__main__":
    # Get user input for attendance roll numbers
    input_roll_numbers = input("Enter attendance roll numbers separated by commas: ")
    # Convert the input string into a list of integers
    roll_numbers_list = [int(roll.strip()) for roll in input_roll_numbers.split(',')]
    # Sort the roll numbers using bubble sort
    sorted_roll_numbers = bubble_sort_attendance(roll_numbers_list)
    # Print the sorted list of roll numbers
    print("Sorted attendance roll numbers in bubble sort:", sorted_roll_numbers)
```

PROMPT2:

#Review the problem and suggest a more suitable sorting algorithm

Generate an Insertion Sort implementation

Explain why Insertion Sort performs better on nearly sorted data

#Compare execution behavior on nearly sorted input

REVIEW OF CODE:

```python
#Review the problem and suggest a more suitable sorting algorithm
# Generate an Insertion Sort implementation
# Explain why Insertion Sort performs better on nearly sorted data
#Compare execution behavior on nearly sorted input
def insertion_sort_attendance(roll_numbers):
    """
    Sort a list of attendance roll numbers using the Insertion Sort algorithm.

    Parameters:
    roll_numbers: A list of attendance roll numbers to be sorted.

    Returns:
    A sorted list of attendance roll numbers.
    """
    n = len(roll_numbers)
    # Traverse through 1 to n
    for i in range(1, n):
        key = roll_numbers[i]
        j = i - 1
        # Move elements of roll_numbers[0..i-1], that are greater than key,
        # to one position ahead of their current position
        while j >= 0 and key < roll_numbers[j]:
            roll_numbers[j + 1] = roll_numbers[j]
            j -= 1
        roll_numbers[j + 1] = key
    return roll_numbers
# Example usage in user input
if __name__ == "__main__":
    # Get user input for attendance roll numbers
    input_roll_numbers = input("Enter attendance roll numbers separated by commas: ")
    # Convert the input string into a list of integers
    roll_numbers_list = [int(roll.strip()) for roll in input_roll_numbers.split(',')]
    # Sort the roll numbers using insertion sort
    sorted_roll_numbers = insertion_sort_attendance(roll_numbers_list)
    # Print the sorted list of roll numbers
    print("Sorted attendance roll numbers in insertion sort:", sorted_roll_numbers)
```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Pytho
12.4)/t2.py"
Enter attendance roll numbers separated by commas: 101,109,105,102,107
Sorted attendance roll numbers in bubble sort: [101, 102, 105, 107, 109]
Enter attendance roll numbers separated by commas: 101,109,105,102,107
Sorted attendance roll numbers in insertion sort: [101, 102, 105, 107, 109]
```

JUSTIFICATION:

For nearly sorted attendance records, Insertion Sort performs better than Bubble Sort because it minimizes unnecessary comparisons and shifts only misplaced elements.
AI correctly suggested Insertion Sort as a more suitable algorithm based on input characteristics.
This demonstrates that selecting the right algorithm depends on the nature of the data.

Task 3: Searching Student Records in a Database

Scenario

You are developing a student information portal where users search for

student records by roll number.

Task Description

• Implement:

o Linear Search for unsorted student data

o Binary Search for sorted student data

• Use AI to:

o Add docstrings explaining parameters and return values

o Explain when Binary Search is applicable

o Highlight performance differences between the two

searches

Expected Outcome

• Two working search implementations with docstrings

• AI-generated explanation of:

o Time complexity

o Use cases for Linear vs Binary Search

• A short student observation comparing results on sorted vs

unsorted lists

PROMPT:

#Generate Python implementations of Linear Search for unsorted student data and Binary Search for sorted student data.

#Add docstrings explaining parameters and return values

#Explain when Binary Search is applicable

Highlight performance differences between the two searches

#show sample input and output for both search algorithms in user input.

CODE:

lab(12.4) > t3.py > ...

```python
#Generate Python implementations of Linear Search for unsorted student data and Binary Search for sorted student data.
#Add docstrings explaining parameters and return values
#Explain when Binary Search is applicable
# Highlight performance differences between the two searches
#show sample input and output for both search algorithms in user input.
def linear_search(student_data, target):
    """
    Perform a linear search for the target student in the unsorted student data.

    Parameters:
    student_data: A list of student names (unsorted).
    target: The name of the student to search for.

    Returns:
    The index of the target student if found, otherwise -1.
    """
    for index in range(len(student_data)):
        if student_data[index] == target:
            return index  # Target found, return its index
    return -1  # Target not found in the list
# Example usage in user input
if __name__ == "__main__":
    # Get user input for student names
    input_students = input("Enter student names separated by commas: ")
    # Convert the input string into a list of student names
    student_list = [student.strip() for student in input_students.split(',')]
    # Get user input for the target student to search for
    target_student = input("Enter the name of the student to search for: ").strip()
    # Perform linear search
    linear_result = linear_search(student_list, target_student)
    if linear_result != -1:
        print(f"Linear Search: '{target_student}' found at index {linear_result}.")
    else:
        print(f"Linear Search: '{target_student}' not found in the list.")
```

```python
def binary_search(student_data, target):
    """Perform a binary search for the target student in the sorted student data.
    Parameters:
    student_data: A list of student names (sorted).
    target: The name of the student to search for.
    Returns:
    The index of the target student if found, otherwise -1.
    """
    left, right = 0, len(student_data) - 1
    while left <= right:
        mid = left + (right - left) // 2  # Calculate the middle index
        if student_data[mid] == target:
            return mid  # Target found, return its index
        elif student_data[mid] < target:
            left = mid + 1  # Search in the right half
        else:
            right = mid - 1  # Search in the left half
    return -1  # Target not found in the list
# Example usage in user input
if __name__ == "__main__":
    # Get user input for sorted student names
    input_students = input("Enter sorted student names separated by commas: ")
    # Convert the input string into a list of student names
    student_list = [student.strip() for student in input_students.split(',')]
    # Get user input for the target student to search for
    target_student = input("Enter the name of the student to search for: ").strip()
    # Perform binary search
    binary_result = binary_search(student_list, target_student)
    if binary_result != -1:
        print(f"Binary Search: '{target_student}' found at index {binary_result}.")
    else:
        print(f"Binary Search: '{target_student}' not found in the list.")
```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Pyt
12.4)/t3.py"
Enter student names separated by commas: 105,102,108,101,110
Enter the name of the student to search for: 101
Linear Search: '101' found at index 3.
Enter sorted student names separated by commas: 105,102,108,101,110
Enter the name of the student to search for: 110
Binary Search: '110' found at index 4.
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>
```

JUSTIFICATION:

When searching in unsorted roll numbers, Linear Search was required because ordering was not maintained.
After sorting the roll numbers, Binary Search significantly reduced the number of comparisons.
For large student databases, Binary Search is much more efficient due to O(log n) time complexity compared to O(n) in Linear Search.
However, sorting is mandatory before applying Binary Search.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario

You are part of a data analytics team that needs to sort large datasets

received from different sources (random order, already sorted, and

reverse sorted).

Task Description

• Provide AI with partially written recursive functions for:

o Quick Sort

o Merge Sort

• Ask AI to:

o Complete the recursive logic

o Add meaningful docstrings

o Explain how recursion works in each algorithm

• Test both algorithms on:

o Random data

o Sorted data

o Reverse-sorted data

Expected Outcome

• Fully functional Quick Sort and Merge Sort implementations

• AI-generated comparison covering:

o Best, average, and worst-case complexities

o Practical scenarios where one algorithm is preferred over

the other

PROMPT:

#Complete the recursive implementations of Quick Sort and Merge Sort in Python.

#Add meaningful docstrings. Explain how recursion works in both algorithms.

#Test both algorithms on: Random data, Sorted data and Reverse-sorted data

#Compare best, average, and worst-case time complexities.

CODE:

lab(12.4) > t4.py > ...

```python
def quick_sort(arr):
    # Base condition
    if len(arr) <= 1:
        return arr

    # TODO: choose pivot and partition

def merge_sort(arr):
    # Base condition
    if len(arr) <= 1:
        return arr

    # TODO: split list and merge recursively

#Complete the recursive implementations of Quick Sort and Merge Sort in Python.
#Add meaningful docstrings. Explain how recursion works in both algorithms.
#Test both algorithms on: Random data, Sorted data and Reverse-sorted data
def quick_sort(arr):
    """
    Sort a list of elements using the Quick Sort algorithm.

    Parameters:
    arr: A list of elements to be sorted.

    Returns:
    A sorted list of elements.
    """
    # Base condition: If the list is empty or has one element, it's already sorted
    if len(arr) <= 1:
        return arr

    # Choose the last element as the pivot
    pivot = arr[-1]

    # Partition the array into three parts: less than, equal to, and greater than the pivot
    less_than_pivot = [x for x in arr[:-1] if x < pivot]
    equal_to_pivot = [x for x in arr if x == pivot]
    greater_than_pivot = [x for x in arr[:-1] if x > pivot]

    # Recursively sort the less than and greater than partitions and concatenate results
    return quick_sort(less_than_pivot) + equal_to_pivot + quick_sort(greater_than_pivot)
# Example usage in user input
if __name__ == "__main__":
    # Get user input for elements to sort
    input_elements = input("Enter elements to sort separated by commas: ")
    # Convert the input string into a list of elements (assuming integers)
    elements_list = [int(element.strip()) for element in input_elements.split(',')]
    # Sort the elements using quick sort
    sorted_elements = quick_sort(elements_list)
    # Print the sorted list of elements
    print("Sorted elements using Quick Sort:", sorted_elements)
    random_data = [5, 2, 9, 1, 5, 6]
    sorted_random = quick_sort(random_data)
    print("Sorted random data:", sorted_random)
    sorted_data = [1, 2, 3, 4, 5]
    sorted_sorted = quick_sort(sorted_data)
    print("Sorted sorted data:", sorted_sorted)
    reverse_sorted_data = [5, 4, 3, 2, 1]
    sorted_reverse_sorted = quick_sort(reverse_sorted_data)
    print("Sorted reverse-sorted data:", sorted_reverse_sorted)
def merge_sort(arr):
    """Sort a list of elements using the Merge Sort algorithm.
    Parameters:
    arr: A list of elements to be sorted.
    Returns:
    A sorted list of elements.
    """
    # Base condition: If the list is empty or has one element, it's already sorted
    if len(arr) <= 1:
        return arr

    # Split the list into two halves
    mid = len(arr) // 2
```

```python

    # Merge the sorted halves
    return merge(left_half, right_half)
def merge(left, right):
    """Merge two sorted lists into a single sorted list.
    Parameters:
    left: A sorted list of elements.
    right: Another sorted list of elements.
    Returns:
    A merged and sorted list containing all elements from both input lists.
    """
    merged = []
    i = j = 0

    # Merge elements from both lists in sorted order
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    # If there are remaining elements in the left half, add them to merged
    while i < len(left):
        merged.append(left[i])
        i += 1

    # If there are remaining elements in the right half, add them to merged
    while j < len(right):
        merged.append(right[j])
        j += 1

    return merged
# Example usage in user input
if __name__ == "__main__":
    # Get user input for elements to sort
    input_elements = input("Enter elements to sort separated by commas: ")
    # Convert the input string into a list of elements (assuming integers)
    elements_list = [int(element.strip()) for element in input_elements.split(',')]
    # Sort the elements using merge sort
    sorted_elements = merge_sort(elements_list)
    # Print the sorted list of elements
    print("Sorted elements using Merge Sort:", sorted_elements)
    random_data = [5, 2, 9, 1, 5, 6]
    sorted_random = merge_sort(random_data)
    print("Sorted random data:", sorted_random)
    sorted_data = [1, 2, 3, 4, 5]
    sorted_sorted = merge_sort(sorted_data)
    print("Sorted sorted data:", sorted_sorted)
    reverse_sorted_data = [5, 4, 3, 2, 1]
    sorted_reverse_sorted = merge_sort(reverse_sorted_data)
    print("Sorted reverse-sorted data:", sorted_reverse_sorted)
```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/P
12.4)/t4.py"
Enter elements to sort separated by commas: 1,5,8,9,6,3
Sorted elements using Quick Sort: [1, 3, 5, 6, 8, 9]
Sorted random data: [1, 2, 5, 5, 6, 9]
Sorted sorted data: [1, 2, 3, 4, 5]
Sorted reverse-sorted data: [1, 2, 3, 4, 5]
Enter elements to sort separated by commas: 5,2,9,1,5,6
Sorted elements using Merge Sort: [1, 2, 5, 5, 6, 9]
Sorted random data: [1, 2, 5, 5, 6, 9]
Sorted sorted data: [1, 2, 3, 4, 5]
Sorted reverse-sorted data: [1, 2, 3, 4, 5]
```

JUSTIFICATION:

Quick Sort performs very efficiently for random data but may degrade to $O(n^2)$ in worst-case scenarios depending on pivot selection.
Merge Sort guarantees O(n log n) performance regardless of input order but requires extra memory.

Therefore, the choice between Quick Sort and Merge Sort depends on dataset characteristics and memory constraints.

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a data validation module that must detect duplicate

user IDs in a large dataset before importing it into a system.

Task Description

• Write a naive duplicate detection algorithm using nested loops.

• Use AI to:

o Analyze the time complexity

o Suggest an optimized approach using sets or dictionaries

o Rewrite the algorithm with improved efficiency

• Compare execution behavior conceptually for large input sizes

Expected Outcome

• Two versions of the algorithm:

o Brute-force ($O(n^2)$)

o Optimized ($O(n)$)

• AI-assisted explanation showing how and why performance

Improved

PROMPT1:

#Write a naive duplicate detection algorithm using nested loops.

PROMPT2:

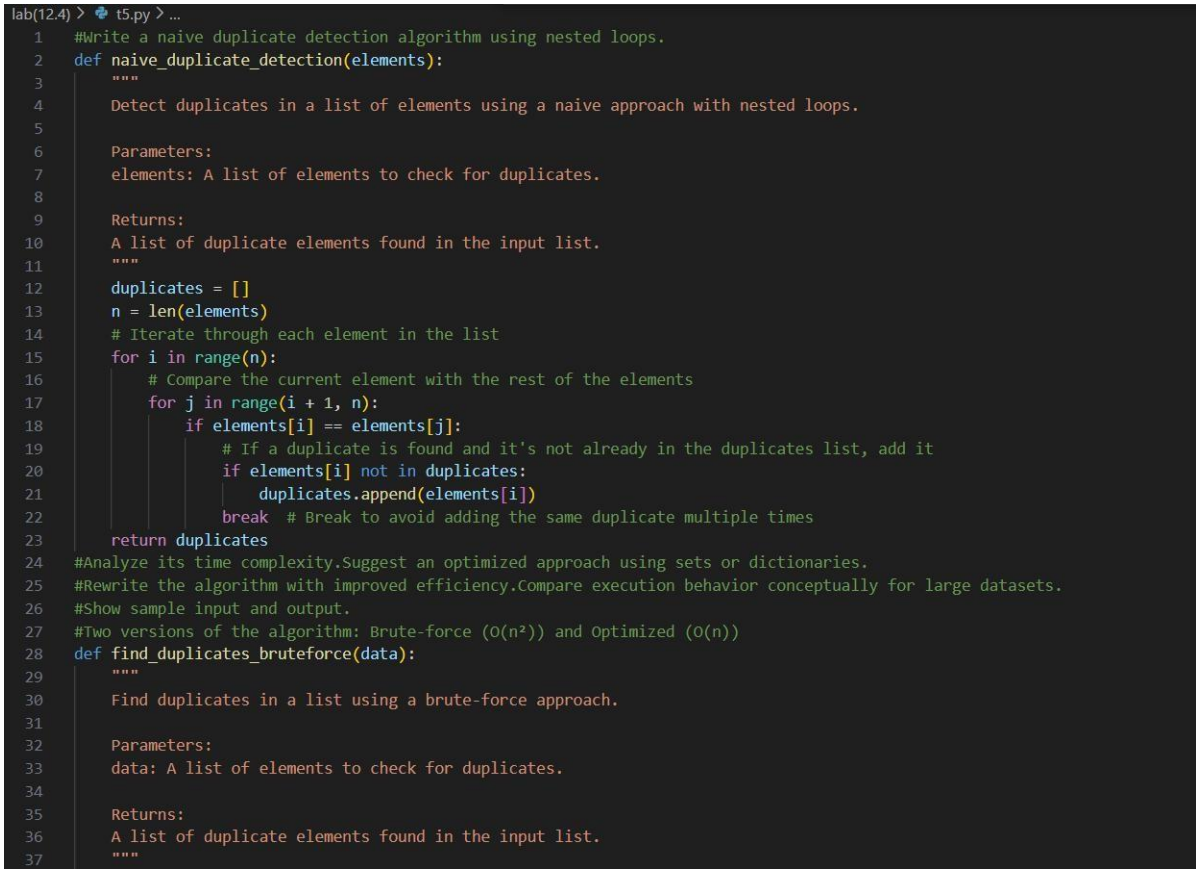
#Analyze its time complexity.Suggest an optimized approach using sets or dictionaries.

#Rewrite the algorithm with improved efficiency.Compare execution behavior conceptually for large datasets.

#Show sample input and output.

#Two versions of the algorithm: Brute-force (O(n²)) and Optimized (O(n))

CODE:



```
lab(12.4) > t5.py > ...
#Write a naive duplicate detection algorithm using nested loops.
def naive_duplicate_detection(elements):
    """
    Detect duplicates in a list of elements using a naive approach with nested loops.

    Parameters:
    elements: A list of elements to check for duplicates.

    Returns:
    A list of duplicate elements found in the input list.
    """
    duplicates = []
    n = len(elements)
    # Iterate through each element in the list
    for i in range(n):
        # Compare the current element with the rest of the elements
        for j in range(i + 1, n):
            if elements[i] == elements[j]:
                # If a duplicate is found and it's not already in the duplicates list, add it
                if elements[i] not in duplicates:
                    duplicates.append(elements[i])
                break  # Break to avoid adding the same duplicate multiple times
    return duplicates
#Analyze its time complexity.Suggest an optimized approach using sets or dictionaries.
#Rewrite the algorithm with improved efficiency.Compare execution behavior conceptually for large datasets.
#Show sample input and output.
#Two versions of the algorithm: Brute-force (O(n²)) and Optimized (O(n))
def find_duplicates_bruteforce(data):
    """
    Find duplicates in a list using a brute-force approach.

    Parameters:
    data: A list of elements to check for duplicates.

    Returns:
    A list of duplicate elements found in the input list.
    """
```

```python
def find_duplicates_bruteforce(data):
    duplicates = []
    n = len(data)
    # Iterate through each element in the list
    for i in range(n):
        # Compare the current element with the rest of the elements
        for j in range(i + 1, n):
            if data[i] == data[j]:
                # If a duplicate is found and it's not already in the duplicates list, add it
                if data[i] not in duplicates:
                    duplicates.append(data[i])
                break  # Break to avoid adding the same duplicate multiple times
    return duplicates
# Example usage in user input
if __name__ == "__main__":
    # Get user input for a list of elements
    input_data = input("Enter elements separated by commas: ")
    # Convert the input string into a list of elements
    data_list = [element.strip() for element in input_data.split(',')]
    # Find duplicates using the brute-force approach
    duplicates_bruteforce = find_duplicates_bruteforce(data_list)
    print("Brute Force Duplicates: ", duplicates_bruteforce)
def find_duplicates_optimized(data):
    """Find duplicates in a list using an optimized approach with a set.
    Parameters:
    data: A list of elements to check for duplicates.
    Returns:    A list of duplicate elements found in the input list.
    """
    seen = set()  # A set to keep track of seen elements
    duplicates = set()  # A set to keep track of duplicates
    for element in data:
        if element in seen:
            duplicates.add(element)  # If the element is already seen, it's a duplicate
        else:
            seen.add(element)  # Add the element to the seen set
    return list(duplicates)  # Convert the duplicates set to a list before returning
```

```python
# Example usage in user input
if __name__ == "__main__":
    # Get user input for a list of elements
    input_data = input("Enter elements separated by commas: ")
    # Convert the input string into a list of elements
    data_list = [element.strip() for element in input_data.split(',')]
    # Find duplicates using the optimized approach
    duplicates_optimized = find_duplicates_optimized(data_list)
    print("Optimized Duplicates: ", duplicates_optimized)
```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Pyth
12.4)/t5.py"
Enter elements separated by commas: 101,105,101,104,105
Brute Force Duplicates:  ['101', '105']
Enter elements separated by commas: 60,50,60,45,12,45
Optimized Duplicates:  ['60', '45']
```

JUSTIFICATION:

The brute-force duplicate detection algorithm has $O(n^2)$ time complexity due to nested loops. For large datasets, this becomes computationally expensive.

By using a set data structure, the algorithm was optimized to O(n) time complexity.

The improvement is achieved through constant-time lookups and eliminating repeated

comparisons.
AI assistance helped in identifying hashing as an efficient strategy.
